



Instituto Superior Técnico
University of Lisbon

Parallel and Distributed Computing

Document Classification Project Report - OMP

Group 17:

João Miguel Tiago – 99986

Maria João Pupo Correia – 86476

Tomás Simões Cândido – 106470

MSc in Computer Science and Engineering
2nd Semester 2025/2026

April 3, 2026

Contents

1	Introduction	2
2	Parallelization approach	2
3	Data decomposition	2
4	Synchronization	2
5	Load Balancing	3
6	Performance results	3
6.1	Results	3
6.2	Discussion	3

1 Introduction

In this stage of the project, we parallelized the document classification algorithm using OpenMP and evaluated the speedup relative to the serial implementation.

SIMD was intentionally not used. Since the serial implementation, used as baseline, is not vectorized, enabling SIMD only in the parallel version would mix two effects (vectorization + threading), making the speedup analysis unfair.

2 Parallelization approach

The dominant cost of the algorithm lies in three inner loops executed at every iteration: (i) accumulating document features sums for each cabinet, (ii) computing new centroids, and (iii) reassigning documents. The entire iterative loop is enclosed in a single `#pragma omp parallel` region to avoid repeated allocations and fork/join overhead. Per-thread accumulators are allocated once inside an `omp single` block, where `omp_get_num_threads()` captures the runtime thread count. Each iteration executes three parallelised steps:

- Phase 1 — Accumulation: each thread accumulates the document vectors of its assigned documents into thread-private per-cabinet partial sums.
- Phase 2 — Cabinets' centroid merge: threads divide the C cabinets among themselves; each thread merges all partial sums for its subset of cabinets and computes the new centroid mean.
- Phase 3 — Reassignment: each thread reassigns its documents to the nearest centroid and records whether any moved, to check for convergence.

Certain tasks, however, remain inherently sequential, including the initial round-robin assignment, memory allocation of `thread_cents` and `thread_cnts` (performed inside an `omp single` region), and convergence control (resetting `changed` and updating it via `atomic write`). These operations are not parallelized either because they must be executed only once or because they involve shared state requiring synchronization.

3 Data decomposition

In phases 1 and 3, work is distributed over documents, with each thread handling a subset of them ($D/nthreads$), whereas in phase 2, the decomposition shifts to the C cabinets, so each thread independently merges all *nthreads* partial sums for its subset of cabinets.

To prevent race conditions during accumulation, the implementation uses thread-local temporary storage: `thread_cents` ($C \times nthreads \times S$) stores partial sums of document feature vectors, and `thread_cnts` ($C \times nthreads$) stores the number of documents contributing to each sum, both per thread and per cabinet.

Each thread writes only to its own slice indexed by $(c * nthreads + tid)$, eliminating the need for locks or atomics during iterations.

4 Synchronization

Because each thread writes only to its own private accumulator slot, phase 1 requires no explicit synchronisation. Phase ordering is ensured via implicit barriers at the end of each `#pragma omp for` loops.

In phase 2, each cabinet is handled by exactly one thread in the merge loop, so no two threads write to the same centroid entry. The implicit barrier at the end of the for loop ensures all centroids are finalised before phase 3 reads them.

Convergence is controlled by the shared variable `changed`. At the beginning of each iteration, one thread resets it using `#pragma omp single`, avoiding conflicts. Then, whenever a thread detects a change, it updates `changed` with an atomic write, so no updates are lost. A final barrier ensures that all threads see the correct value before the next iteration.

5 Load Balancing

Load balancing is addressed by parallelizing the dominant loops over documents using `#pragma omp for schedule(static)`, ensuring that each thread receives an approximately equal number of iterations. Since each document requires a similar amount of computation (fixed-size inner loops over features and cabinets), this results in a naturally balanced workload with minimal scheduling overhead. Additionally, the use of thread-local accumulators (`thread_cents` and `thread_cnts`) avoids contention between threads during accumulation, allowing each thread to progress independently without delays caused by synchronization.

6 Performance results

Experiments were run on a machine with 6 physical cores and 12 hardware threads. Each experiment was executed three times to account for variability in execution time caused by CPU scheduling, memory/cache effects, and other processes running on the machine. The values presented in Table 1 correspond to the arithmetic mean of these three runs.

6.1 Results

Implement. (n ^o of threads)	Target (linear)	ex1000-50000-200			ex200-400000-10			ex10-200000-100		
		time (s)	speedup	efficiency	time (s)	speedup	efficiency	time (s)	speedup	efficiency
serial	1.0	76.9	1.0	-	67.2	1.0	-	109.6	1.0	-
// 1-thread	1.0	77.0	1.00	1.00	145.8	0.46	0.46	104.7	1.05	1.05
// 2-threads	2.0	38.9	1.98	0.99	78.4	0.86	0.43	58.5	1.87	0.94
// 4-threads	4.0	21.3	3.61	0.90	46.7	1.44	0.36	32.5	3.37	0.84
// 6-threads	6.0	16.1	4.78	0.80	33.9	1.98	0.33	24.4	4.49	0.75
// 8-threads	8.0	16.9	4.55	0.57	39.7	1.69	0.21	29.2	3.75	0.47
// 12-threads	12.0	13.5	5.70	0.48	31.3	2.15	0.18	21.8	5.03	0.42
// 16-threads	16.0	14.2	5.42	0.34	34.9	1.93	0.12	23.4	4.68	0.29
// 24-threads	24.0	13.7	5.61	0.23	33.1	2.03	0.08	21.8	5.03	0.21

Table 1: Execution time, theoretical (linear) speedup, measured speedup (serial / n threads), and parallel efficiency (speedup / number of threads).

6.2 Discussion

In an ideal setting, speedup would scale linearly with the number of threads. This would require the algorithm to be fully parallel, with no sequential sections or overheads. In practice, this is not (and never could be) the case. According to Amdahl’s Law, the presence of even small serial fractions limits the maximum achievable speedup. In this implementation, sequential components include the initial round-robin assignment, memory allocation of `thread_cents` and `thread_cnts` (performed inside

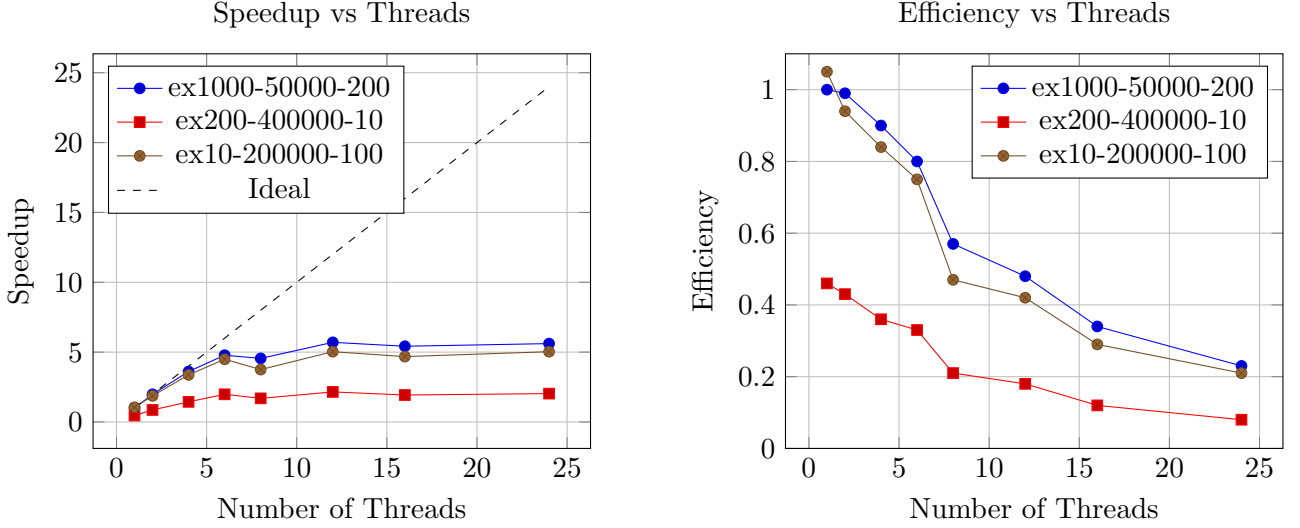


Figure 1: Speedup and efficiency as a function of the number of threads for the evaluated instances.

an `omp single region`), and convergence control (resetting changed and updating it through synchronization). Additionally, barriers between accumulation, centroid computation, and reassignment phases introduce synchronization points where threads must wait, further reducing scalability.

To better assess scalability, in Table 1 speedup is reported relative to the serial implementation, and efficiency is computed as the ratio between speedup and the number of threads. This highlights how effectively additional threads are used. The results deviate from linear speedup as expected. For `ex1000-50000-200`, speedup scales well up to 6 threads up to when it increases near linear rate as reported in Figure 1-right ($4.78\times$, efficiency 80%), but then saturates, reaching a maximum of $5.70\times$ (efficiency 48%) at 12 threads, with no further gains at higher thread counts. A similar trend is observed for `ex10-200000-100`, which reaches a speedup of $5.03\times$ (efficiency 42%) at 12 threads. In contrast, `ex200-400000-10` achieves only $2.15\times$ speedup (efficiency 18%), showing significantly poorer scalability.

The OpenMP execution with one thread is slower than the serial version in some cases (e.g., 145.8s vs 67.2s for `ex200-400000-10`). This is due to parallel overhead, including thread management, repeated entry into parallel regions, synchronization constructs, and the use of thread-local buffers. These costs are absent in the serial implementation and become dominant when no parallelism is exploited.

The variation across inputs reflects the balance between computation and memory access. Instances with higher dimensionality (`ex1000-50000-200` and `ex10-200000-100`) achieve better performance because the inner loops over features increase arithmetic intensity, allowing threads to perform more computation per memory access. In contrast, `ex200-400000-10`, with only 10 features, is memory-bound: threads spend most of the time accessing shared arrays (`docs`, `centroids`), leading to memory bandwidth saturation and poor efficiency. This explains the sharp drop in efficiency as the number of threads increases (down to 8% at 24 threads). In all cases, the primary source of idle time identified by VTune was threads stalled at implicit barriers, waiting for slower threads to finish their assigned chunk. To address this, we experimented with `schedule(guided)` and `schedule(dynamic)` on the reassignment loop, which improved CPU utilization but increased overall execution time due to the additional scheduling overhead. Since the goal is minimizing wall-clock time rather than maximizing utilization, `schedule(static)` was retained as the better practical choice.

Hardware constraints further limit scaling. Up to 6 threads execution maps well to physical cores.

The major efficiency drop is observed in Figure1-right between 6 and 12 threads since hyperthreading leads to contention for execution units, caches, and memory bandwidth, reducing per-thread efficiency. Beyond this point, additional threads provide little benefit, as memory bandwidth becomes the dominant bottleneck. The plateau observed in Figure1-left beyond 12 threads across all instances reflects this limitation.

Although SIMD vectorization was not applied, the inner loops over document features are highly vectorisable. Applying SIMD only to the parallel version would bias the comparison, as the serial baseline would not benefit from the same optimization.

Overall, efficiency is bounded by the serial fraction of the code (as described by Amdahl's Law), synchronization overhead, and memory bandwidth limitations. The implementation achieves good scalability for compute-intensive workloads, but becomes memory-bound for low-dimensional inputs, where increasing the number of threads yields diminishing returns.